

AI 编程可维护性技能实战：让 AI 成为你的代码质量守门员

土豆1250 2026-04-11 👁 181 ⌚ 阅读19分钟

关注

适合人群：已经在用 Claude Code、Cursor 等 AI 编程工具的开发人员，希望把 AI 从"写代码工具"升级成"代码质量合伙人"的同学。

为什么你需要了解它？（Why）

你有没有遇到过这种场景：

打开一个新的 AI 对话框，开始愉快地写代码。AI 刷刷刷地给出方案，你复制粘贴，一切看起来很好。

但三天后，你打开这段代码，发现它像一坨意大利面——嵌套了四五层的 `if-else`，变量叫 `data1`、`data2`、`temp`、`result`，注释为零。

这不是 AI 的问题，这是空白石板（Blank Slate）问题：每次开新对话，AI 对你的项目一无所知——不知道你的命名规范，不知道你的架构原则，更不知道你们团队眼中的"好代码"长什么样。

于是有人开始把规范粘贴到每个对话里，有人写了超长的 System Prompt，有人每次都要重新解释一遍"我们用 ES Modules，不用 CommonJS".....重复劳动，消耗心力。

Agent Skills（智能体技能） 就是为了解决这个问题而生的。它相当于给 AI 装上了一套"项目记忆"和"专业工具箱"，让代码简化、代码审查这类专业工作变成可复用的能力。

而要安装这些技能，最便捷的工具就是 Vercel 出品的 `skills` CLI。

它到底是什么？（What）

核心概念

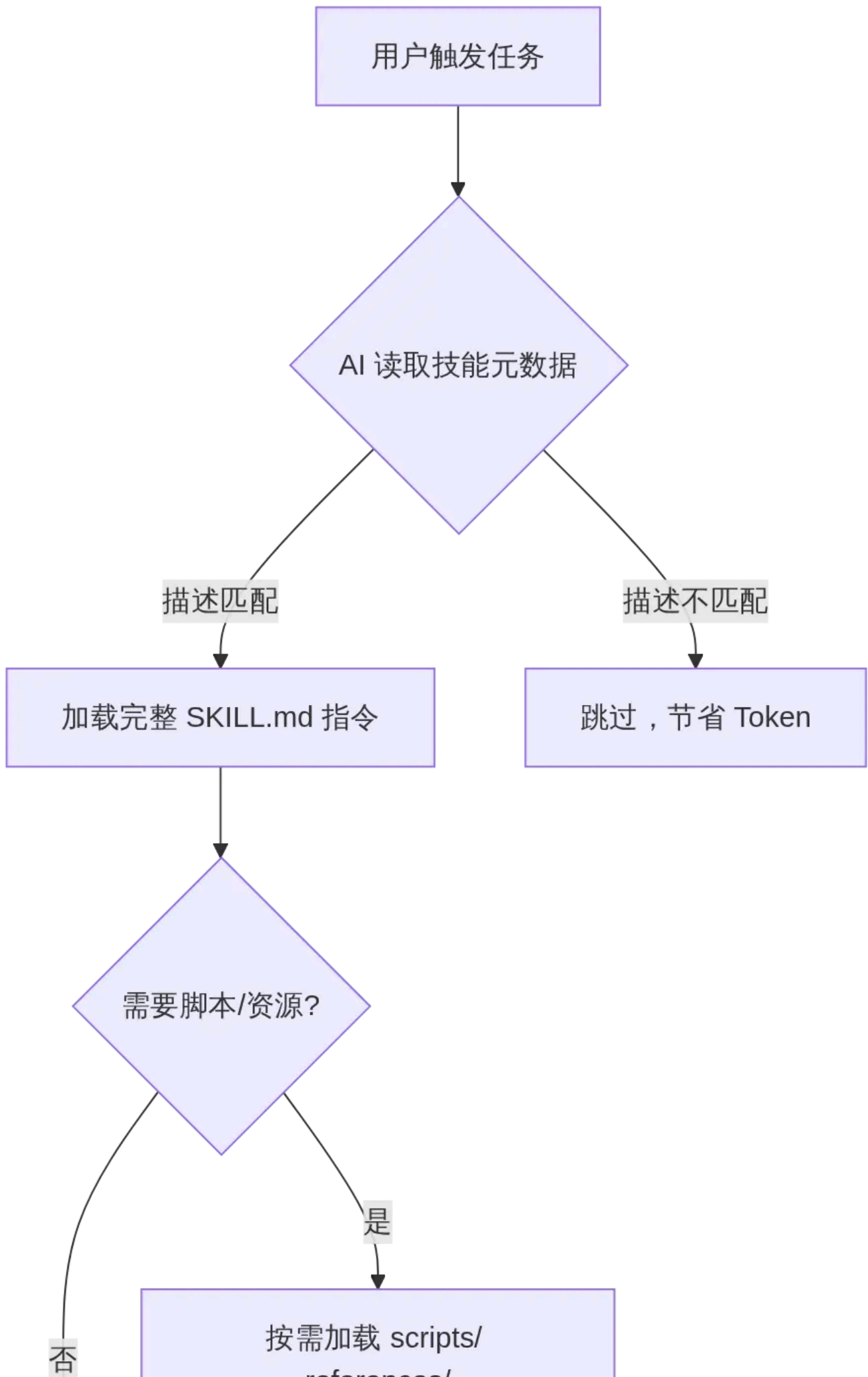
Agent Skills 是一种以 **SKILL.md** 为核心的技能包格式。每个技能就是一个文件夹，里面放着：

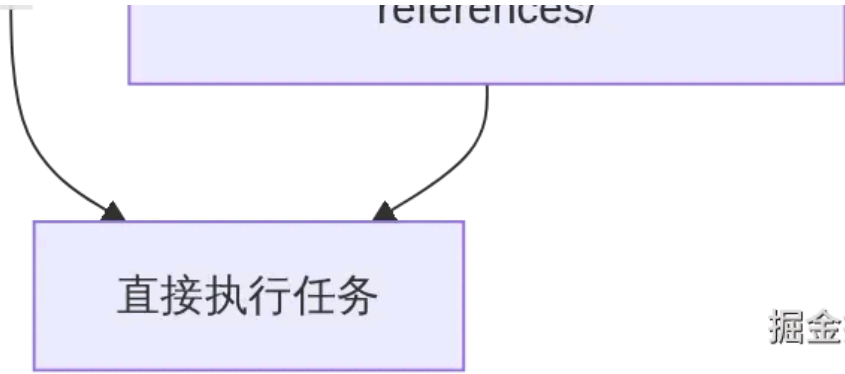
- **SKILL.md**：技能的"说明书"，包含 YAML 元数据和 Markdown 指令
- **scripts/**（可选）：自动化脚本
- **references/**（可选）：更长的参考文档、检查清单
- **assets/**（可选）：模板、配置文件

用一个比喻来说：如果 AI 是一位新来的员工，**AGENTS.md**（常驻规则）是你的"员工手册"，而 **Skills** 则是"专项培训教材"——只在需要时取出来用，不浪费上下文窗口。

工作原理

技能系统采用**渐进式披露（Progressive Disclosure）**策略，巧妙地避免了 Context 爆炸：





掘金技术社区 @ 土豆1250

- 1. 第一层：AI 只读取 `name` + `description` 元数据，决定是否激活
- 2. 第二层：激活后才加载完整的 `SKILL.md` 指令体
- 3. 第三层：只在指令要求时才加载辅助脚本和资源

这意味着即使你安装了几十个技能，每次对话也不会被撑爆。

被动上下文 vs 主动技能

类型	放哪里	触发方式	适合内容
被动上下文	<code>AGENTS.md</code> / <code>CLAUDE.md</code>	始终生效	安全守则、格式规范、"永不绕过认证"
主动技能	<code>skills/</code> 文件夹	按需激活	专项工作流，如代码简化、安全审计

怎么用？（How）

快速上手：安装 skills CLI

Vercel 出品的 `skills` CLI 是目前最主流的技能安装工具。虽然可以直接 `npx skills` 使用，但强烈建议先全局固定安装 **1.4.0** 版本——原因后面会详细解释，简单说就是新版在 Cursor Windows 环境下有已知 Bug。

第一步：全局安装 1.4.0

bash

1

`npm i -g skills@1.4.0`

安装后，所有命令直接用 `skills`，不需要加 `npx`，也不会被 npm 自动拉取新版本：

bash

体验AI代码助手

代码解读

复制代码

```
1 # 安装某个技能包 (以 Anthropic 官方技能库为例)
2 skills add anthropics/skills
3
4 # 查看某个仓库里有哪些技能，不安装
5 skills add anthropics/skills --list
6
7 # 只安装其中某几个技能
8 skills add anthropics/skills --skill code-simplifier --skill security-review
9
10 # 全局安装 (跨项目生效)
11 skills add -g anthropics/skills
12
13 # 指定目标 Agent (如 Claude Code 或 Cursor)
14 skills add anthropics/skills --agent claude-code
15
16 # 安装完后查看已安装技能
17 skills list
18
19 # 搜索社区技能
20 skills find typescript
```

第二步：把用法规则写入 AGENTS.md，让 AI 也遵守

光你自己记得用 `skills` 还不够——如果 AI 帮你执行安装时，它可能习惯性地写出 `npx skills`，一不小心触发有 bug 的最新版。把这条规则写进项目（或全局）的 `AGENTS.md`，AI 就会永远遵守：

markdown

体验AI代码助手

代码解读

复制代码

```
1 <!-- ~/.claude/AGENTS.md 或项目根目录的 AGENTS.md -->
2
3 ## skills CLI 使用规范
4
5 - 操作 Agent Skills 时，必须使用全局安装的 `skills` 命令
6 - 禁止使用 `npx skills` (会拉取最新版，在 Cursor Windows 环境下有已知 Bug)
7 - 正确示例: `skills add xixu-me/skills --skill skills-cli -g`
8 - 错误示例: `npx skills add xixu-me/skills --skill skills-cli -g`
```

这样无论你在哪个项目、哪次对话里让 AI 帮你装技能，它都只会用你机器上那个稳定的 1.4.0。

小提示： `skills` CLI 支持多种来源——GitHub 简写、完整 URL、GitLab、本地路径，甚至 git SSH 地址，用起来就像 npm 装包一样自然。

先搞清楚：两套技能分发系统

在介绍具体技能前，有必要先厘清一个容易混淆的概念：AI 编程技能存在**两套并行**的分发体系，覆盖范围不同：

系统	安装方式	支持工具	代表来源
Claude.ai 插件 (Plugin)	在 claude.com/plugins 一键安装	仅 Claude Code	Anthropic 官方
开放技能 CLI (SKILL.md)	<code>skills add</code> 命令行安装	Claude Code、Cursor、Gemini CLI 等	任何开发者

Anthropic 有两个代码质量的**官方 Plugin**，经过验证，仅能在 Claude Code 中使用；而 **skills CLI 生态**则是面向所有 AI 编辑器的开放格式。此外还要纠正一个常见误区：
`anthropics/skills` 开源仓库里装的是文档处理技能（docx/pdf/pptx/xlsx）和设计类工具，不含代码质量技能——这个误区在网上广泛流传，请注意甄别。

场景一：Code Simplifier——代码简化

官方版本（**Claude Code 专用**）：claude.com/plugins/cod...，Anthropic Verified，178K 安装

这个技能的定位很精准：介于「实现完成」和「代码审查」之间的那个步骤，专门负责清理刚写完的代码，在审查之前把可读性先拉上来。在 Claude Code 中，它会在你每次实现代码后自动运行。

Claude Code 用户：在 claude.com 直接安装官方版

进入 claude.com/plugins/cod... 点击安装即可。

Cursor / 其他编辑器用户：通过 **skills CLI** 安装同类技能

经 `skills find` 实际验证，skills CLI 生态中最可信的版本来自 Sentry 工程团队：

bash

 体验AI代码助手

 代码解读

复制代码

```
1 skills add -g getsentry/skills --skill code-simplifier
```

技能做什么：

- 消除不必要的嵌套和复杂度
- 删除冗余代码和过度抽象
- 改善变量名和函数名
- 用更清晰的结构替换嵌套三元表达式
- 应用项目一致的编码标准

触发方式：

arduino

 体验AI代码助手

 代码解读

复制代码

```
1 "simplify this component"
2 "clean up this code"
3 "refactor for clarity"
4 "优化一下这个函数的可读性"
```

实战效果对比：

简化前：

javascript

 体验AI代码助手

 代码解读

复制代码

```
1 // 嵌套三元地狱
2 const status = user ? (user.active ? (user.admin ? 'admin' : 'user') : 'inactive') : 'gu
```

简化后：

javascript

 体验AI代码助手

 代码解读

复制代码

```
1 const getUserStatus = (user) => {
2   if (!user) return 'guest';
3   if (!user.active) return 'inactive';
4   return user.admin ? 'admin' : 'user';
5 };
```

```
6
7  const status = getUserStatus(user);
```

场景二：Code Review——代码审查

官方版本（Claude Code 专用）：[claude.com/plugins/cod...](https://claude.com/plugins/codereview)，Anthropic Verified，212K 安装

这是目前安装量最高的代码审查插件。它调度五个专项子 Agent 并行工作，分别负责：CLAUDE.md 规范合规检查、Bug 检测、git 历史上下文分析、历史 PR 评论回顾、代码注释核查。每条问题有 0-100 的置信度评分，默认只输出置信度 80 以上的问题，大幅减少误报噪声。

Claude Code 用户：直接安装官方版，用 `/code-review` 触发

进入 [claude.com/plugins/cod...](https://claude.com/plugins/codereview) 安装后，在任意 PR 分支上运行 `/code-review`，结果会以 GitHub 评论形式直接发布。

Cursor / 其他编辑器用户：通过 skills CLI 安装

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1  # 方案 A: Sentry 版 (通用, 无需额外账号)
2  skills add -g getsentry/skills --skill code-review
3
4  # 方案 B: CodeRabbit 版 (与 CodeRabbit 平台深度集成, 1.5K 安装)
5  skills add -g coderabbitai/skills --skill code-review
```

检查维度：

检查维度	具体内容
Bug 风险	潜在的空指针、边界条件、异步竞争
安全漏洞	XSS、SQL 注入、不安全的依赖使用
代码质量	圈复杂度、重复代码、函数职责单一性
项目规范	命名约定、文件结构、注释要求

触发方式：

arduino

体验AI代码助手 代码解读 复制代码

```
1 "review the changes in this PR"
2 "check this file for issues"
3 "对最近的改动做一次代码审查"
```

场景三：Security Review——安全审计

官方版本：无。Anthropic 目前没有官方的 Security Review 插件，此类技能完全来自社区。skills CLI 生态中，Sentry 工程团队的版本最为完整可靠：

bash

体验AI代码助手 代码解读 复制代码

```
1 skills add -g getsentry/skills --skill security-review
```

触发词覆盖广，基于置信度分级输出，针对注入、XSS、认证、授权、加密等常见漏洞类别逐一检查。

触发方式：

arduino

体验AI代码助手 代码解读 复制代码

```
1 "security review"
2 "find vulnerabilities"
3 "check for security issues"
4 "audit security"
5 "OWASP review"
```

Sentry 还提供两个专项安全技能（按需额外安装）：

bash

体验AI代码助手 代码解读 复制代码

```
1 # GitHub Actions  workflow安全审计 (pwn request、expression injection 等 CI/CD 漏洞)
2 skills add -g getsentry/skills --skill gha-security-review
3
4 # Django  专项安全审计 (IDOR、对象权限、多租户隔离)
5 skills add -g getsentry/skills --skill django-access-review
```

场景四：skills CLI 技能——让 AI 学会管理自己的技能库

一句话定位：这个技能让 AI 自己知道怎么安装、更新、查找技能，而不是每次都靠你手动敲命令。

有没有想过：你在用一个技能教 AI 如何写代码，但 AI 自己不知道"技能"是什么东西。每次你说"帮我装个 code-reviewer 技能"，它就一脸茫然。

安装 **skills CLI** 技能之后，AI 就掌握了整套技能管理工作流，可以自主执行安装、搜索、更新等操作。

技能来源：[xixu-me/skills](#)（社区贡献，非 Vercel 官方），45.6K 安装量，是目前该类别下最广泛使用的版本。

▼ bash

体验AI代码助手

代码解读

复制代码

```
1 # 已全局安装 skills@1.4.0 的情况下，直接用 skills 命令
2 skills add xixu-me/skills --skill skills-cli -g
```

触发场景：当你说"帮我安装 XX 技能"、"搜索一下有没有 React 相关技能"、"更新所有技能"时自动激活。

⚠ Cursor Windows 用户必读：为什么必须全局安装 1.4.0

这是一个真实存在的已知 Bug，踩坑无数人。

问题根因：从 v1.4.1 开始，skills CLI 将全局技能安装到 `~/.agents/skills/`，而 Cursor 实际上只从 `~/.cursor/skills/` 加载技能。结果就是：技能装了，但 Cursor 看不见——技能面板显示 **"Agents: not linked"**（未链接）。

这个问题在 Windows 环境下尤其严重，因为 symlink 的处理方式在 Windows 上和 Linux/macOS 不同。（相关 Issue：[#421](#)、[#537](#)）

根本解法：全局固定安装 1.4.0，不再依赖 npx

▼ bash

体验AI代码助手

代码解读

复制代码

```
1 # 一次安装, 永久生效
2 npm i -g skills@1.4.0
3
4 # 此后所有操作直接用 skills, 不加 npx
5 skills add -g anthropics/skills --skill code-simplifier
6 skills add -g xixu-me/skills --skill skills-cli
```

v1.4.0 会直接将技能安装到 `~/.cursor/skills/`，Cursor 能正确加载。全局安装后不会被 npx 自动升级，彻底规避问题。

如果已经用新版 npx 装了技能，出现 `unlinked` 怎么救？

```
▼ bash 体验AI代码助手 代码解读 复制代码
1 # 先全局安装好 1.4.0
2 npm i -g skills@1.4.0
3
4 # 卸载旧的 (从错误目录清理), 重装到正确位置
5 skills remove code-simplifier -g
6 skills add -g anthropics/skills --skill code-simplifier
```

关注修复进度： Vercel 团队正在处理此问题（PR [#464](#)、[#694](#)），后续版本修复后可升级，目前 Windows + Cursor 的最佳选择就是全局固定 **1.4.0**。

场景五：agent-browser——给 AI 装上眼睛和手

技能来源：[vercel-labs/agent-browser](https://vercel-labs.com/agent-browser)，Vercel 官方出品

代码写好了，但要验证效果，你还是得自己打开浏览器手动点？有没有想过让 AI 直接帮你操作浏览器——打开页面、截图、填表、抓数据？

关于 web-access 技能的说明

社区中有一个同类技能 [eze-is/web-access](#)（直接连接你现有 Chrome 的 CDP，天然携带登录态），功能相当强大，但属于个人开发者作品。相比之下，**agent-browser** 是 **Vercel 官方维护**的，文档完整、更新稳定、与 vercel-labs 生态打通，对于大多数场景更值得信赖。如果你有特殊需求（复用现有 Chrome 会话、中文界面提示），可以额外了解 [eze-is/web-access](#)。

agent-browser 就是给 AI 装上"浏览器控制权"的技能。它基于 Chrome DevTools Protocol (CDP)，启动一个受控的 Chromium 实例，让 AI 能够真实操作浏览器，而不是只靠纯语言推理猜测页面状态。

安装核心技能：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # 前提：先全局安装 agent-browser CLI 工具
2 npm i -g agent-browser
3
4 # 安装核心浏览器技能（教 AI 如何使用）
5 npx skills@1.4.0 add vercel-labs/agent-browser --skill agent-browser
```

agent-browser 的四步核心工作流：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # AI 会按这个流程操作浏览器：
2 # 1. 导航到目标页面
3 agent-browser open https://example.com
4
5 # 2. 截取快照，获取页面可交互元素（返回 @e1, @e2 这样的引用）
6 agent-browser snapshot -i
7
8 # 3. 用引用与元素交互（点击、填表、选择）
9 agent-browser click @e3
10 agent-browser fill @e5 "hello@example.com"
11
12 # 4. 操作后再次截取快照，确认状态变化
13 agent-browser snapshot -i
```

四个场景，四个专项技能：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # 场景一：基础浏览器自动化（导航、截图、表单、数据抓取）
2 npx skills@1.4.0 add vercel-labs/agent-browser --skill agent-browser
3
4 # 场景二：自动化 QA 探索测试，产出 Bug 报告 + 截图
5 # 触发词："dogfood example.com" / "QA 一下登录页面"
6 npx skills@1.4.0 add vercel-labs/agent-browser --skill dogfood
7
8 # 场景三：自动化 Electron 应用（VS Code、Slack、Figma 等桌面 App）
9 npx skills@1.4.0 add vercel-labs/agent-browser --skill electron
```

```
10
11 # 场景四：在 Vercel Sandbox 微型 VM 中运行 (适合 CI/CD 场景)
12 npx skills@1.4.0 add vercel-labs/agent-browser --skill vercel-sandbox
```

实战触发示例：

arduino  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 "打开 localhost:3000，截图给我看看登录页面的样子"
2 "帮我测试一下注册流程，看看有没有问题"
3 "dogfood http://localhost:3000 - 重点测试结账页"
4 "对比 staging 和 production 的首页，看哪里不一样"
```

代码质量场景应用：

结合 Code Reviewer 技能，可以实现一个端到端的"写代码 → 自动运行 → 浏览器截图验证 → AI 审查结果"闭环：

markdown  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 1. AI 写好新功能
2 2. Code Simplifier 自动清理代码
3 3. agent-browser 打开本地开发服务器，截图验证 UI
4 4. dogfood 技能自动探索页面，找出潜在 bug
5 5. Code Reviewer 对改动做最终代码审查
```

这一套流程跑完，你只需要最后看一眼报告——剩下的全是 AI 干的。

场景六：在 Cursor 中的实战

Cursor 从 2025 年起全面支持 Agent Skills 格式，在 Cursor 中使用技能有两种方式：

方式 A：安装到项目（团队共享）

bash  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 # 在项目根目录执行，技能会保存到 .cursor/skills/
2 skills add getsentry/skills --skill code-simplifier --skill code-review --skill security
```

这样整个团队拉代码时都能用同一套技能，告别"在我电脑上是好的"的噩梦。

方式 B：全局安装（个人专用）

bash

体验AI代码助手

代码解读

复制代码

```
1 # 一次安装 Sentry 全套代码质量技能
2 skills add -g getsentry/skills --skill code-simplifier --skill code-review --skill secur
```

安装后，在 Cursor 的 Composer 模式（Agent 模式）中，直接描述任务，Cursor 会自动激活匹配的技能。

三层质量管控体系（推荐架构）：

arduino

体验AI代码助手

代码解读

复制代码

```
1 ~/.cursor/skills/           ← 第一层：全局偏好（所有项目生效）
2   └─ code-simplifier/
3   └─ code-reviewer/
4
5 repo-root/.cursor/skills/   ← 第二层：项目规范（团队共享）
6   └─ security-review/
7   └─ custom-style-guide/
8
9 services/api/.cursor/skills/ ← 第三层：模块规则（服务级）
10  └─ api-conventions/
```

配合 AGENTS.md 使用效果翻倍：

markdown

体验AI代码助手

代码解读

复制代码

```
1 <!-- repo-root/AGENTS.md -->
2 # 项目规范
3
4 - 使用严格 TypeScript (strict: true)
5 - 所有 public 函数必须有 JSDoc 注释
6 - 单文件不超过 300 行
7 - 禁止 barrel exports (index.ts 统一导出)
8 - 新功能必须先写 spec，再写代码
```

场景七：在 Claude Code 中的实战

Claude Code 是命令行原生的 AI 编程工具，与 Agent Skills 配合最为紧密。

安装技能到 Claude Code：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # 安装 Sentry 代码质量三件套, 指定 claude-code agent
2 skills add -g getsentry/skills --agent claude-code --skill code-simplifier --skill code-
3
4 # 安装 skills-cli 技能, 让 AI 自己会管理技能
5 skills add -g xixu-me/skills --agent claude-code --skill skills-cli
```

推荐的开发工作流（质量导向）：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # 第一步: 进入 Plan 模式, AI 先理解需求, 不急着写代码
2 /plan 实现用户登录功能, 要求支持 JWT 和 session 两种模式
3
4 # 第二步: AI 生成实现方案, 确认后开始编码
5 /build
6
7 # 第三步: Code Simplifier 自动触发, 清理复杂度
8 # (或手动触发)
9 simplify the recently changed auth module
10
11 # 第四步: Code Reviewer 审查
12 review the changes for bugs and security issues
13
14 # 第五步: 安全审计 (可选, 针对关键模块)
15 run security review on src/auth/
```

团队协作技巧：

把技能锁定到特定版本，防止技能更新破坏团队工作流：

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 # 安装特定 commit 版本 (更稳定)
2 skills add https://github.com/getsentry/skills/tree/main
3
4 # 检查并升级已安装技能
5 skills check
6 skills update
```

自定义技能：打造你的团队专属"代码守门员"

标准技能不够用？自己造一个，只需要 `npx skills init`：

bash

体验AI代码助手

代码解读

复制代码

```
1 # 初始化一个新技能模板
2 npx skills init my-code-review
3
4 # 生成的结构
5 my-code-review/
6 └─ SKILL.md
```

一个团队定制的代码审查技能示例：

markdown

体验AI代码助手

代码解读

复制代码

```
1 ---
2 name: my-code-review
3 description: "审查代码是否符合团队规范。当用户说「代码审查」、「review」、「检查代码」时使用。"
4 ---
5
6 # 团队代码审查规范
7
8 ## 必须检查项
9
10 1. **命名规范**: 组件用 PascalCase, 工具函数用 camelCase, 常量用 UPPER_SNAKE_CASE
11 2. **文件大小**: 单文件不超过 300 行, 超过则建议拆分
12 3. **错误处理**: 所有异步操作必须有 try-catch, 且错误信息要有意义
13 4. **测试覆盖**: 新增业务逻辑必须附带单元测试
14
15 ## 不允许的模式
16
17 - `any` 类型 (TypeScript 项目)
18 - `console.log` 出现在非调试代码中
19 - 硬编码的 URL 或密钥
20
21 ## 输出格式
22
23 按优先级分类输出: [CRITICAL] > [WARNING] > [SUGGESTION]
24 每条包含: 文件路径 + 行号 + 问题描述 + 修改建议
```

发布到 GitHub 后，团队成员一行命令即可安装：

bash

体验AI代码助手

代码解读

复制代码


```
1 npx skills add your-org/your-skills-repo
```

最佳实践

-  **description** 当路由规则写，不要写成标题。好的描述是："简化代码复杂度，消除嵌套。当用户说「优化」、「simplify」、「重构」、「代码太乱」时使用。" 描述写不好，AI 永远不知道该激活哪个技能。
-  被动规则放 **AGENTS.md**，专项技能放 **Skills**。格式规范、安全守则是"随时在线"的；而代码简化、安全审计是"按需召唤"的。分开放，上下文窗口才不会爆。
-  项目级技能提交到 **Git** 仓库，让整个团队共享同一套 AI 行为规范，杜绝"每个人用 AI 写出来的代码风格完全不同"的混乱局面。
-  不要把所有规则塞进一个超长 **SKILL.md**。善用 **references/** 文件夹存放详细文档，**SKILL.md** 只留核心指令，让渐进式披露机制发挥作用。
-  不要盲目安装陌生技能，特别是带 **scripts/** 目录的——那些脚本是真实执行的，和跑 npm 包一样需要审查。
-  全局安装 **skills@1.4.0**，并在 **AGENTS.md** 中禁止 AI 使用 **npx skills**。v1.4.1 之后有安装目录 Bug，**npx** 每次都会拉最新版踩坑。一次 **npm i -g skills@1.4.0** 搞定，配合规则让 AI 也只用全局的 **skills** 命令，彻底消除隐患。

常见误区与避坑指南

误区	正确理解	解决方案
"技能会自动修复所有代码问题"	技能是指导 AI 的工作流，不是魔法按钮	把技能理解为"专业同事的工作手册"，AI 仍需执行和判断
"AGENTS.md 和 Skills 选一个就够了"	两者定位不同，互补使用才完整	常驻规范 → AGENTS.md；专项工作流 → Skills
"技能格式因工具而异，不通用"	标准化 SKILL.md 格式已跨平台通用	同一个技能可用于 Claude Code、Cursor、Gemini CLI
"anthropics/skills 里有代码质量技能"	该仓库只含文档处理和设计类技能	Code Simplifier/Review 官方插件仅限 Claude Code，其他编辑器用 getsentry/skills
"Security Review 也有 Anthropic 官方版"	Anthropic 目前没有官方 Security Review 插件	使用 getsentry/skills --skill security-review，Sentry 出品可信度高
"Code Simplifier 会改变代码逻辑"	它只改"怎么写"，不改"做什么"	简化前后行为完全一致，但可读性大幅提升
"description 写短点节省 token"	description 是激活触发器，太短会失灵	description 要包含触发场景和关键词，稍微长一点值得
"Windows Cursor 用最新 skills CLI 没问题"	v1.4.1+ 在 Cursor Windows 下有 unlinked Bug	npm i -g skills@1.4.0 全局固定版本，在 AGENTS.md 中禁止 AI 用 npx skills
"agent-browser 只是个截图工具"	它是完整的浏览器自动化框架，可交互、抓数据、跑 QA	结合 dogfood 技能，能实现自动化探索测试并输出 Bug 报告

进阶资源

- [claude.com/plugins/cod...](#) — Anthropic 官方 Code Simplifier 插件（178K 安装，Claude Code 专用）
- [claude.com/plugins/cod...](#) — Anthropic 官方 Code Review 插件（212K 安装，Claude Code 专用）
- [anthropics/skills](#) — Anthropic 开源技能仓库（文档处理、设计类，非代码质量类）
- [getsentry/skills](#) — Sentry 工程团队，skills CLI 中代码质量三件套的最佳来源
- [coderabbitai/skills](#) — CodeRabbit 官方技能，code-review 专业版

- [Vercel Agent Skills 完整文档](#)
- [skills CLI 官方文档](#)
- [vercel-labs/agent-skills](#) — Vercel 官方技能集
- [Cursor Agent 最佳实践](#)
- [Agent Experience 最佳实践 \(Marmelab\)](#)
- [Claude Code 插件库](#)
- [vercel-labs/agent-browser](#) — Vercel 官方浏览器自动化技能，含 dogfood/electron/slack
- [xixu-me/skills](#) — skills-cli 技能来源（社区，45.6K 安装）
- [eze-is/web-access](#) — 社区版浏览器访问技能，直连现有 Chrome
- [skills CLI Windows unlinked Bug 追踪](#) — 了解 1.4.0 版本锁定原因

小结

Agent Skills 是 AI 编程工具进化的一个关键节点——它把"一次性提示词"变成了可版本化、可共享、可复用的团队资产。

核心五件套：

1. `npm i -g skills@1.4.0` + **AGENTS.md** 规则 — 全局固定版本，再用规则让 AI 也只用 `skills` 命令，Windows Cursor 用户彻底告别 unlinked 错误
2. **Code Simplifier + Code Reviewer** — 守住可维护性的"实现后"和"审查前"两道关口
3. **Security Review** — 安全审计最后一公里，8 步扫描覆盖主流漏洞类型
4. **skills CLI 技能** — 让 AI 自己学会管理技能库，自动安装升级不再靠你手敲
5. **agent-browser** — 给 AI 装上浏览器控制权，从"写代码"到"验证结果"全程自动化

最后说一句：代码可维护性不是写完之后才想起来的事，而是从第一行就该建立的习惯。让 AI 帮你守住这个习惯，比靠自律靠谱多了。

标签：

LLM

AI编程

话题：

AI 编程

本文收录于以下专栏



AI大模型

专栏目录

AI与大模型相关内容

3 订阅 · 5 篇文章

订阅

上一篇

LangGraph TypeScript 版入门与实践

评论 0



登录 / 注册

即可发布评论！



暂无评论数据

目录

收起 ^

为什么你需要了解它？（Why）

它到底是什么？（What）

核心概念

工作原理

被动上下文 vs 主动技能

怎么用？（How）

快速上手：安装 skills CLI

先搞清楚：两套技能分发系统

场景一：Code Simplifier——代码简化

场景二：Code Review——代码审查

场景三：Security Review——安全审计

场景四：skills CLI 技能——让 AI 学会管理自己的技能库

⚠️ Cursor Windows 用户必读：为什么必须全局安装 1.4.0

场景五：agent-browser——给 AI 装上眼睛和手

场景六：在 Cursor 中的实战

场景七：在 Claude Code 中的实战

自定义技能：打造你的团队专属"代码守门员"

最佳实践

常见误区与避坑指南

进阶资源

小结

精选内容

AI 正在压垮 GitHub

王若风 · 192阅读 · 0点赞

Gemini Pro 的失败证明 AGI 路线的严重泡沫

龙哥AI · 76阅读 · 0点赞

大模型应用开发工程师面试指南——从入门到通关，拿下高薪Offer

Cosolar · 67阅读 · 1点赞

DeepSeek 的 Vision 能力要来了吗？

龙哥AI · 89阅读 · 1点赞

大模型应用开发 极简入门路线

马艳泽 · 77阅读 · 0点赞

为你推荐

AI时代代码质量提升实战指南：别让效率成为质量的敌人

不被AI替代的BOT | 4月前 | 👁 346 | 👍 点赞 | 💬 评论

程序员 人工智能

AI 代码编程助手真的有用吗

XCaptaino | 1年前 | 👁 159k | 👍 16 | 💬 8

前端 后端 人工智能

前端开发者的 AI 时代生存指南：大模型如何重塑岗位要求与技能

__不想说话__ | 2月前 | 2.3k | 40 | 5

前端 人工智能 面试

AI编程规模化实践：从1到100的工程化之道

码事漫谈 | 5月前 | 160 | 2 | 评论

后端

Superpowers 技术指南：让 AI 编程助手拥有超能力

遇见邹 | 2月前 | 4.4k | 6 | 2

人工智能 AI编程 测试

AI前端开发：加速你的职业晋升之路

用户963774906708 | 1年前 | 99 | 点赞 | 评论

前端

【GitHub每日速递 20260107】pg - aiguide：让AI写出更优PostgreSQL代码，大幅提升数...

piggy侠 | 3月前 | 104 | 点赞 | 评论

GitHub

一天一个开源项目（第3篇）：Superpowers - 让 AI 编程助手拥有超能力的工作流框架

冬奇Lab | 3月前 | 998 | 1 | 评论

开源

让AI成为你的专家：Claude Skills 完全指南

小杨技术笔记 | 3月前 | 464 | 点赞 | 评论

人工智能

AI 代码重构完全指南：利用 LLM 提升代码质量与可维护性

兆子龙 | 9天前 | 27 | 3 | 评论

Java JavaScript

2026年AI编程革命：程序员会被取代吗？出路在哪里？

用户736249950554 | 1月前 | 94 | 点赞 | 评论

算法

AI 原生软件工程的可观测性与可控制性（万字长文）

DevLake技术小组 | 20天前 | 34 | 点赞 | 评论

人工智能 开源

超越SMART——让AI成为你的超级思考伙伴

欧阳尚成 | 8月前 | 291 | 点赞 | 评论

人工智能

人机协奏曲：当代码成为对话，程序员成为导演

邵齐 | 4月前 | 218 | 5 | 评论

前端

Vibe Coding：让AI替你写代码，你只管躺着赚钱！

WildBlue | 11月前 | 7.2k | 55 | 19

Cursor 前端 Claude

